

LLMs and Automata: 把 RNN 的 hidden state 当成自动机状态来审问

图文讲义版：用原 PPT 作为视觉锚点，按“概念故事线 + 直觉解释 + 考试速记”整理。

autoregressive LM 用 chain rule 给字符串赋概率 → representation-based LM 把前缀压成 hidden representation → RNN 递归更新 hidden state → RNN recogniser 把最终 state 二值化为 accept/reject → formalism 用可识别语言类 $L(F)$ 来比较 → FSM 只有有限离散状态，识别 regular languages → RNN state 理论上是连续无限空间 → two-stack PDA 等价于 Turing machine → 用实数编码 stack 与 configuration → unbounded precision RNN 可模拟 2-PDA，因此具备 Turing-level power

Why formal study of LLMs?

- Opening the black box of LLMs: what are the underlying computations they do?
- We can better understand the limitations of LLMs
- Find connections to previous models may help deriving new methodology for LLMs

这节课一句话

这节课用 formal language 的眼光打开神经语言模型黑箱：RNN 一边像有限状态自动机逐符号读入并更新状态，一边又因为连续 hidden state 在理想无限精度下能把两个 stack 藏进实数小数位，从而模拟 Turing machine。

1. 从 formal study 到 Autoregressive LM: 先问模型怎样给字符串打分

What are language models?

- Let $y = y_1 \cdots y_n$
- An autoregressive language model defines $p(y)$

$$p(y) = p(\text{EOS} | y) \prod_{t=1}^n p(y_t | y_{<t})$$

Nothing new! Any distribution over strings can be written that way

Food for thought: Why is that the case? The chain rule

人话直觉

formal study 不是给 LLM 套数学滤镜，而是先把黑箱问成一台清楚的机器：它读什么、记什么、状态怎么变。语言模型的第一层答案很朴素——给一句话打分就像一边读一边下注，每个词的概率乘起来，最后再赌 EOS。

精确定义 / 机制：本讲用 formal languages 和 automata theory 分析 neural language models 的计算能力。起点是 autoregressive LM：给定字符串 $y = y_1 \cdots y_n$ ，把完整字符串概率写成每个 token 在前缀条件下的概率乘积，并包含结束符 EOS。slide 上强调这并不神秘：任何字符串分布都可以这样分解，因为概率论的 chain rule 允许把联合概率拆成条件概率的乘积。representation-based LM 进一步把前缀 $y_{<t}$ 映射为 $h(y_{<t})$ ，再用 softmax 输出 $p(y_t | y_{<t})$ 。

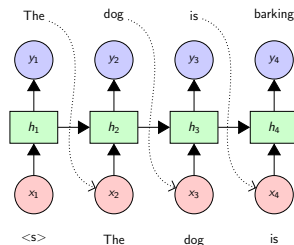
$$p(y) = p(\text{EOS} | y) \prod_{t=1}^n p(y_t | y_{<t}), \quad p(y_t | y_{<t}) = \text{softmax}(Eh(y_{<t}))_{y_t}$$

考试问法：常考“为什么 formal study LLMs”和“LM 如何给句子赋概率”。答题先说 formal methods 能刻画模型底层计算与限制，再写 chain rule，并说明 $h(\text{prefix}) + \text{softmax}$ 是实现条件概率的一种方式。

2. RNN LM: hidden state 是会滚动更新的前缀摘要

RNNs as language models

If we want to turn an RNN into a language model, we add another layer that takes as input h_t and returns a probability distribution over the vocabulary



Every y_t is fed as an x_{t+1} in the next step

人话直觉

n-gram 像只看窗口里的最近几个词；RNN 像带着一本不断压缩的笔记本，每读一个 token 就把旧笔记和新 token 合并成新笔记。理论上，这本笔记可以含有整个前缀的信息。

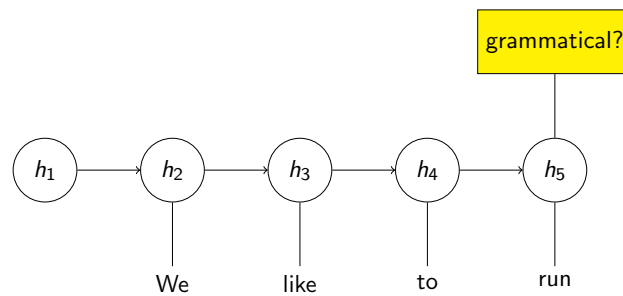
精确定义 / 机制： RNN 对输入序列 $x_1, \dots, x_n$ 递归更新 hidden state: $h_{t+1} = f(h_t, x_t)$ 。本讲聚焦 Elman RNN，形式为 $h_{t+1} = \sigma(Uh_t + Vr(x_t) + b)$ ，其中 $r(x_t)$ 是 token embedding， U 、 V 、 b 是参数。作为 LM 时，RNN 用当前 h_t 经过输出层和 softmax 预测 y_t ，并把生成出的 y_t 作为下一步输入 x_{t+1} 。与 n-gram 相同，它们都通过条件概率连乘给整句赋概率；关键差别是 n-gram 历史窗口固定，而 RNN 的 hidden state 是由整个前缀确定性更新来的。

$$h_{t+1} = f(h_t, x_t) = \sigma(Uh_t + Vr(x_t) + b)$$

考试问法：若问“RNN LM 与 n-gram 的联系和区别”，答：二者都乘条件概率；n-gram 只依赖固定长度历史，RNN 用确定性 hidden state 总结前缀。若问为什么不用 sum out hidden state，答：给定输入前缀，RNN state 是确定的，不像隐变量模型那样需要对可能 hidden states 求和。

3. 从 generator 到 recogniser: RNN 也可以像自动机一样接受/拒绝

RNNs as Recognisers



- Apply a softmax or some other activation unit to binarise the state after encoding into *grammatical* or *not grammatical*
- Given an RNN, its language is defined as the set of all strings it classifies as “grammatical”

人话直觉

同一个 RNN，换个问题就换个身份：做 LM 时它像预测下一个词的播报员；做 recogniser 时它像门口保安，读完整个字符串后看最终 hidden state，决定“grammatical”还是“not grammatical”。

精确定义 / 机制：Finite State Machine 由有限状态集合 Q 、初始状态 q_0 、alphabet Σ 、转移函数 $\delta(q,y)=q'$ 和接受状态集合 F 组成。FSM 逐符号读入字符串，最终落在 F 中就接受。RNN recogniser 与此相似：它逐步消费输入并更新 h_t ，读完整串后用 softmax、threshold 或其他二值分类器把最终状态映射成 accept/reject。于是，一个具体 RNN 定义了一个 language：它接受的所有字符串集合。

$$\delta(q,y) = q', \quad L(A) = \{x \in \Sigma^* : A \text{ accepts } x\}$$

考试问法：常考“FSM 定义”和“RNN 如何作为 recogniser”。FSM 要列 $Q, q_0, \Sigma, \delta, F$ ；RNN recogniser 要说最终 hidden state 被二值化，语言是所有被判为 grammatical/accepted 的字符串。

4. 用 L(F) 比较形式系统：连续 state space 让 RNN 出现分叉

State Space in RNNs

- In principle, the state space for RNNs is infinite, not only infinite, but continuous
- Can we use this fact somehow to simulate really complex algorithms?

We will test RNNs in two cases: the case of unbounded precision and computation and the case of bounded computation and finite precision

Food for thought: What do you think the outcome will be?

人话直觉

比较机器强弱，不看它们长得像不像，而看它们能筛出哪些字符串集合。FSM 像有限个抽屉，用完就没了；RNN 的 hidden state 像连续尺子，理论上有无限细刻度——但前提是你真的能无限精细地读写它。

精确定义 / 机制：给定一个 formalism F ，例如所有 FSM 或所有作为 acceptor 的 RNN；其中每个具体实例 $A \in F$ 都定义一个语言 $L(A)$ 。整个形式系统的能力由 $L(F) = \{L(A) \mid A \in F\}$ 刻画，两个形式系统等价当且仅当语言类相同。FSM 的语言类正是 regular languages。RNN 与 FSM 都逐步读取输入并更新状态，但 FSM 状态有限离散，RNN hidden state 通常是连续向量；因此后半讲分成两个假设：unbounded precision/computation 下，连续 state 可被当成无限精密内存；bounded precision/computation 下，RNN 会退回有限状态式限制。本蓝图聚焦可考的 unbounded 构造。

$$L(F) = \{L(A) \mid A \in F\}, \quad F \equiv F' \iff L(F) = L(F')$$

考试问法：若问“如何 characterise a formalism through its languages”，答 $L(F)$ 定义；若问“RNN 与 FSM 差别”，答 FSM 是有限离散 state，RNN 是连续 state，理论能力取决于是否允许 unbounded precision/computation。

5. Two-stack PDA: 两个栈为什么等价于 Turing machine

Two-stack Pushdown Automaton

An equivalent formalism to Turing machines

Rather than having an infinite tape, we have two stacks. A two-stack PD automaton has:

- Q - a finite set of states
- Σ - a finite alphabet
- Γ - a finite stack alphabet (for both stacks)
- δ - a transition function in $Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \times \Gamma \rightarrow Q \times \Gamma^* \times \Gamma^*$.
The transition outputs a new state and strings to replace the top of the two stacks based on the current input symbol and the top symbols of the two stacks
- $q_0 \in Q$ - an initial state
- $q_A \in Q$ - an accepting state
- $q_R \in Q$ - a reject state

人话直觉

Turing machine 有一条可左右移动的无限纸带；two-stack PDA 用两个栈把纸带从读写头处劈开：左边内容放一个栈，当前格和右边内容放另一个栈。读写头向左或向右移动，本质上就是在两个栈之间 pop/push。

精确定义 / 机制： two-stack pushdown automaton 是与 Turing machine 等价的形式系统。它包含有限状态集 Q 、输入 alphabet Σ 、stack alphabet Γ 、转移函数 δ 、初始状态 q_0 、接受状态 q_A 和拒绝状态 q_R 。其转移依赖当前控制状态、可选输入符号以及两个 stack 的顶部符号，输出新状态和用于替换两个栈顶的字符串。因为两个 stack 可以模拟 Turing tape，所以如果 RNN 能模拟 two-stack PDA，就能模拟 Turing machine。

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \times \Gamma \rightarrow Q \times \Gamma^* \times \Gamma^*$$

考试问法：若问“为什么 two-stack PDA 重要”，答：one-stack PDA 只提供单栈记忆，而 two-stack PDA 可模拟 Turing tape；证明 RNN 模拟 2-PDA 就等于证明它在理想假设下具备 Turing completeness。

6. 把 stack 编码进实数: push/pop 变成小数点搬运

Performing Stack Operations

Stack operations can be performed by manipulating a real number that represents the whole stack:

- Popping $y_N = 0 : 10 \times 0.\gamma_N\gamma_{N-1}\dots\gamma_1 - 1$
- Popping $y_N = 1 : 10 \times 0.\gamma_N\gamma_{N-1}\dots\gamma_1 - 3$
- Pushing $y = 0 : \frac{1}{10} \times 0.\gamma_N\gamma_{N-1}\dots\gamma_1 + \frac{1}{10}$
- Pushing $y = 1 : \frac{1}{10} \times 0.\gamma_N\gamma_{N-1}\dots\gamma_1 + \frac{3}{10}$

人话直觉

这是全讲最漂亮也最危险的一步：把整个栈写成一个小数，栈顶放在最靠近小数点的位置。pop 就像把最前面一位拿掉再整体左移；push 就像先整体右移，再把新符号塞到小数点后第一位。

精确定义 / 机制：讲义假设 stack alphabet 可取二进制 $\Gamma = \{0, 1\}$ 。栈 $y_1 \dots y_N$ 被编码成实数 $0.\gamma_N\gamma_{N-1}\dots\gamma_1$ ，其中 $\gamma_i = 1$ 表示 $y_i = 0$ ， $\gamma_i = 3$ 表示 $y_i = 1$ 。不用 0/1 直接作为小数位，是为了避免 trailing zeros 带来的歧义。若栈顶编码为 1，pop 可写作 $10s-1$ ；若栈顶编码为 3，pop 可写作 $10s-3$ 。push 0 可写作 $s/10+1/10$ ；push 1 可写作 $s/10+3/10$ 。这个编码要求任意长小数位都可可靠保存，因此依赖 unbounded precision。

$$s = 0.\gamma_N\gamma_{N-1}\dots\gamma_1, \quad \gamma_i = \begin{cases} 1, & y_i = 0 \\ 3, & y_i = 1 \end{cases}$$

考试问法：常考“RNN 如何模拟 stack / 为什么这个构造不现实”。答题必须说：stack 内容藏在实数小数位中，乘除 10 和加减常数实现 push/pop；但任意长 stack 需要无限精度，实际浮点机器会丢信息。

7. Configuration encoding: hidden vector 是整台 2-PDA 的快照

How to encode a “configuration” of a 2-stack PDA machine?

- A vector in the RNN will be in $\mathbb{R}^d$ where $d = 2 + 3 + 3 + N$ where N is the number of states in the Turing machine
- How to encode a configuration of a 2-stack PDA using a vector in the RNN h ?
 - h_1 is the first stack encoded as before
 - h_2 is the second stack encoded as before
 - h_3, h_4 and h_5 encode the top element of the first stack (remember the alphabet is $\{0, 1\}$ and we need one more slot for denoting an empty stack!)
 - h_6, h_7 and h_8 encode the top element of the second stack
 - The rest of the vector (positions 9 to $8 + N$) indicate which state the PDA is currently in

人话直觉

只编码两个栈还不够，就像拍机器照片不能只拍内存，还要拍控制面板。RNN hidden vector 需要同时记住：两个栈长什么样、两个栈顶是什么、PDA 现在处在哪个离散状态。

精确定义 / 机制：一个 2-PDA 的 configuration 是“当前世界状态”：包括两个 stack 的内容、两个 stack 顶部符号以及当前控制状态。讲义构造一个 $d=2+3+3+N$ 维 RNN hidden vector: h_1 和 h_2 存两个 stack 的实数编码; h_3, h_4, h_5 one-hot 编码第一个栈顶为 0、1 或 empty; h_6, h_7, h_8 编码第二个栈顶; 剩余 h_9 到 h_{8+N} one-hot 表示 PDA 当前状态。若能把每一步 configuration 正确更新为下一步 configuration，就能模拟整台 2-PDA。

$$d = 2 + 3 + 3 + N$$

考试问法：若问“RNN hidden state 如何编码 PDA configuration”，按三块答：两个实数维度存 stack contents，两个 3 维 one-hot 存 top-of-stack， N 维 one-hot 存 current state。强调这是构造性证明，不是训练过程自然学到。

8. 最终结论: unbounded RNN 是 Turing complete, 但别误读成现实万能

Summary: Turing Machines and RNNs

- Turing machines can be simulated using two-stack automata
- Two-stack automata can be simulated using RNNs with unbounded precision and computation
- The state in the RNN encodes the two stacks and the state in the two-stack automaton (a “configuration”)
- Therefore: **RNNs with unbounded precision and computation are as powerful as Turing machines!**

人话直觉

这节课的结论像一句带星号的广告语: RNN 可以和 Turing machine 一样强——星号写着“在无限精度和理想计算条件下”。去掉星号, 就会把理论存在性误读成现实可训练性。

精确定义 / 机制: 构造路线是: Turing machines 可由 two-stack automata 模拟; two-stack automata 的 configuration 可编码进 RNN hidden state; RNN 参数 U, V, b 可被构造出来, 使状态更新按照 2-PDA 的转移规则推进。因此, 在 unbounded precision and computation 假设下, RNN 与 Turing machine 一样强, 能实现任何可计算函数。slide 26 还强调这不是训练得到的结论, 而是理论上手工设置参数的 constructive proof。实际 RNN 还受有限精度、有限计算、优化失败、数据有限和泛化问题限制。

考试问法: 若考“RNNs are Turing complete 如何理解”, 最稳答案是: 在理想无限精度/计算下, RNN 可用连续 hidden state 编码两个 stack 并模拟 two-stack PDA; 因为 2-PDA 等价于 Turing machine, 所以 RNN 具备 Turing-level expressivity。但这不说明实际训练出的 RNN 一定能学会或稳定执行任意算法。

考试速记版

- Autoregressive LM 的句子概率来自 chain rule: 逐 token 条件概率乘起来, 再乘 EOS。
- Representation-based LM: 先把 prefix 映射为 $h(\text{prefix})$, 再用 softmax 预测下一个 token。
- Elman RNN 更新式是 $h_{t+1} = \sigma(Uh_t + Vx_t + b)$, hidden state 是前缀摘要。
- RNN LM 与 n-gram 都给字符串赋概率; n-gram 看固定窗口, RNN state 由整个前缀确定性更新。
- RNN 作为 recogniser: 读完整串后把最终 hidden state 二值化为 accept/reject。
- FSM = $Q, q_0, \Sigma, \delta, F$; 它识别 regular languages。
- 形式系统 F 的能力用 $L(F) = \{L(A) | A \in F\}$ 表示; 语言类相同则形式系统等价。
- RNN hidden state 是连续空间; 无限精度下可当无限细内存, 有限精度下结论会变。

- Two-stack PDA 等价于 Turing machine: 两个 stack 可模拟 tape 左右两侧。
- Stack 编码用 1 表示 0、3 表示 1，避免 trailing zeros 歧义。
- Push/pop 可用实数算术实现：乘 10 / 除 10 / 加减 1 或 3。
- Turing completeness 是构造性存在证明，不等于实际训练一定学到，也不等于有限精度机器万能。

一段稳妥考场回答

本讲用 formal languages 分析 RNN/LLM 的计算能力。Autoregressive LM 先用 chain rule 将字符串概率写成 $p(\text{EOS}|y)$ 与每个 $p(y_t|y_{<t})$ 的乘积；RNN 通过确定性更新 hidden state $h_{t+1} = f(h_t, x_t)$ 来总结前缀并预测下一个 token。若把 RNN 看成 recogniser，它和 FSM 一样逐符号读取输入并更新状态，最终把 hidden state 映射成 accept/reject，因此可以用其接受的语言 $L(A)$ 来研究表达力。FSM 的状态有限离散，只识别 regular languages；RNN 的 hidden state 是连续向量，在 unbounded precision and computation 的理想假设下，可以把 two-stack PDA 的两个 stack 编码进实数小数位，并用乘除 10 与加减常数模拟 push/pop，同时用额外维度 one-hot 编码栈顶和当前 PDA 状态。因为 two-stack PDA 等价于 Turing machine，所以这种理想化 RNN 可模拟 Turing machine，具有 Turing-level expressive power。但这只是构造性理论结果，不说明实际有限精度、有限数据和训练得到的 RNN 一定能稳定学会任意算法。